

Actividad 4: Interfaz Gráfica de Usuario
Programación Orientada a Objetos

Ejercicios 6.4 a 6.8 – Excepciones y lectura de archivos
Ejercicios de programación orientada a objetos con Java y UML – Leonardo Bermón Angarita

Facultad de Ingeniería

Universidad Nacional de Colombia
Docente: Walter Hugo Arboleda Mazo

Ximena Muñoz Lara
Noviembre de 2026

Contenido

Ejercicio 6.4 – Excepciones (pág. 400)..... 3
Interfaz de usuario:..... 5
Diagrama de Clase:..... 5
Casos de uso:..... 6
URL en GitHub:..... 6
Ejercicio 6.5 – Vendedor (pág. 406)..... 7
Interfaz de usuario:..... 9
Diagrama de Clase:..... 10
Casos de uso:..... 11
URL en GitHub:..... 11
Ejercicio 6.6 – Cálculos numéricos (pág. 412)..... 12
Interfaz de usuario:..... 14
Diagrama de Clase:..... 15
Casos de uso:..... 15
URL en GitHub:..... 16
Ejercicio 6.7 – Equipo de maratón de programación (pág. 418)..... 17
Interfaz de usuario:..... 21
Diagrama de Clase:..... 22
Casos de uso:..... 23
URL en GitHub:..... 23
Ejercicio 6.8 – Lectura de archivos (pág. 427)..... 24
Interfaz de usuario:..... 26
Diagrama de Clase:..... 26
Casos de uso:..... 27
URL en GitHub:..... 27
URL del repositorio en GitHub:..... 27

Ejercicio 6.4 – Excepciones (pág. 400)

Enunciado: ¿Cuál es el resultado de la ejecución del método main del siguiente programa? Determinar qué se imprime en pantalla. El programa tiene dos bloques try que generan excepciones (una división por cero y el uso de un objeto nulo), cada uno con sus bloques catch y finally. En esta versión la ejecución y sus mensajes se muestran en una interfaz gráfica de usuario.

Archivo 1: prueba_excepciones.py

```
class PruebaExcepciones:
    def __init__(self):
        # aqui se guardan los mensajes que se van mostrando
        self.mensajes = []

    def primer_bloque(self):
        try:
            self.mensajes.append("Ingresando al primer try")
            cociente = 10000 / 0  # se lanza una excepcion
            self.mensajes.append("Después de la división")  # nunca se ejecuta
        except ZeroDivisionError:
            self.mensajes.append("División por cero")
        finally:
            # el finally siempre se ejecuta
            self.mensajes.append("Ingresando al primer finally")

    def segundo_bloque(self):
        try:
            self.mensajes.append("Ingresando al segundo try")
            objeto = None
            objeto.toString()  # se lanza una excepcion
            self.mensajes.append("Imprimiendo objeto")  # nunca se ejecuta
        except ArithmeticError:
            # esta excepcion no es de este tipo, no entra aca
            self.mensajes.append("División por cero")
        except Exception:
            self.mensajes.append("Ocurrió una excepción")
        finally:
            self.mensajes.append("Ingresando al segundo finally")

    def ejecutar(self):
        self.mensajes = []
        self.primer_bloque()
        self.segundo_bloque()
        return self.mensajes
```

Archivo 2: ventana_excepciones.py

```
import tkinter as tk
from prueba_excepciones import PruebaExcepciones
```

```
class VentanaExcepciones(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Excepciones")
        self.resizable(False, False)

        tk.Label(self, text="Resultado de la ejecución del programa:", anchor="w").place(x=20, y=15,
width=300, height=23)

        # cuadro de texto donde se muestran los mensajes
        self.texto = tk.Text(self, state="disabled")
        self.texto.place(x=20, y=45, width=340, height=150)

        self.boton_ejecutar = tk.Button(self, text="Ejecutar", command=self.ejecutar)
        self.boton_ejecutar.place(x=20, y=210, width=100, height=25)
        self.boton_limpiar = tk.Button(self, text="Limpiar", command=self.limpiar)
        self.boton_limpiar.place(x=130, y=210, width=100, height=25)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 380) // 2
        y = (self.winfo_screenheight() - 255) // 2
        self.geometry(f"380x255+{x}+{y}")

    # evento del boton Ejecutar
    def ejecutar(self):
        prueba = PruebaExcepciones()
        mensajes = prueba.ejecutar()
        self.texto.config(state="normal")
        self.texto.delete("1.0", tk.END)
        self.texto.insert(tk.END, "\n".join(mensajes))
        self.texto.config(state="disabled")

    # evento del boton Limpiar
    def limpiar(self):
        self.texto.config(state="normal")
        self.texto.delete("1.0", tk.END)
        self.texto.config(state="disabled")
```

Archivo 3: principal_excepciones.py

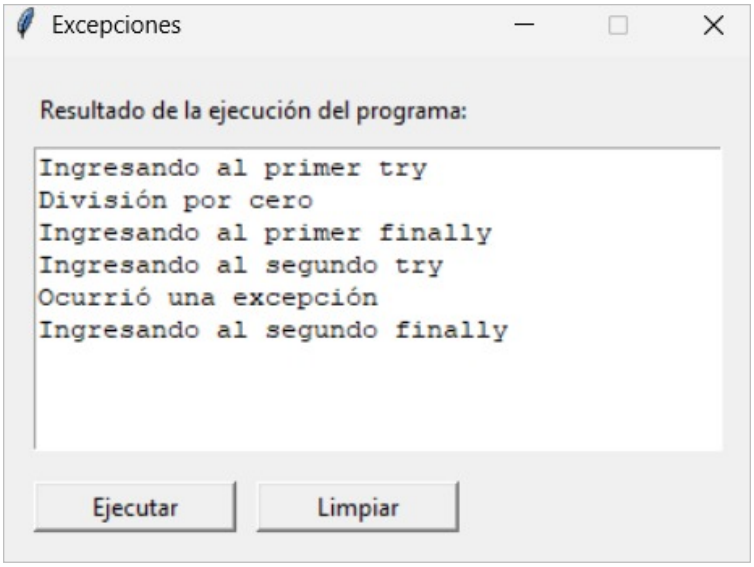
```
from ventana_excepciones import VentanaExcepciones


class Principal:
    def main():
        ventana = VentanaExcepciones()
        ventana.mainloop()


if __name__ == "__main__":
```

```
    Principal.main()
```

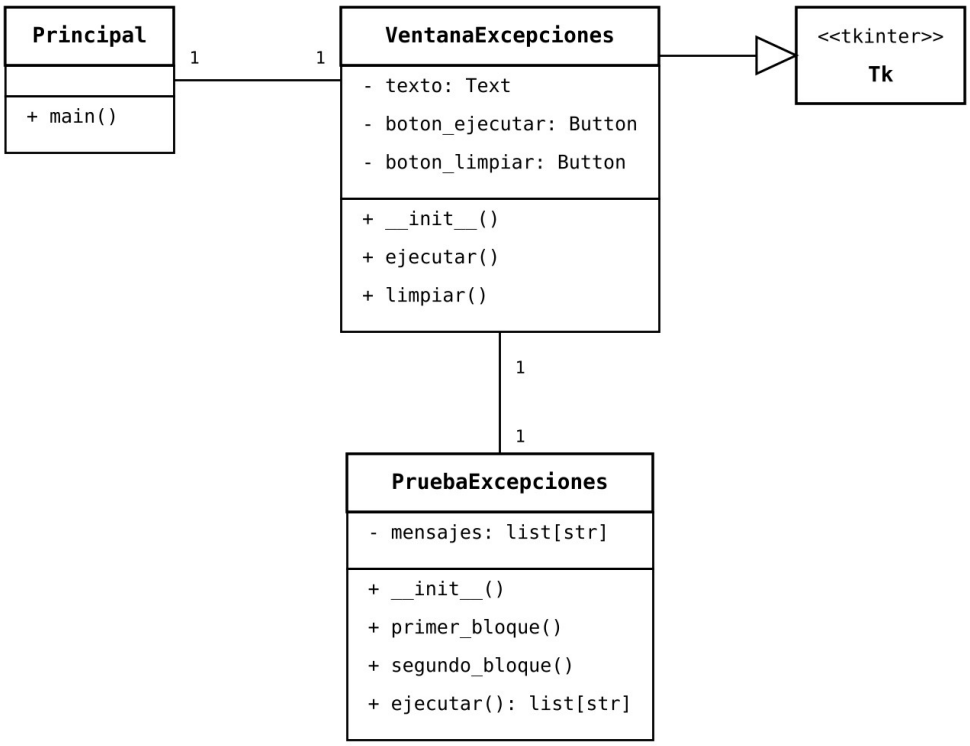
Interfaz de usuario:



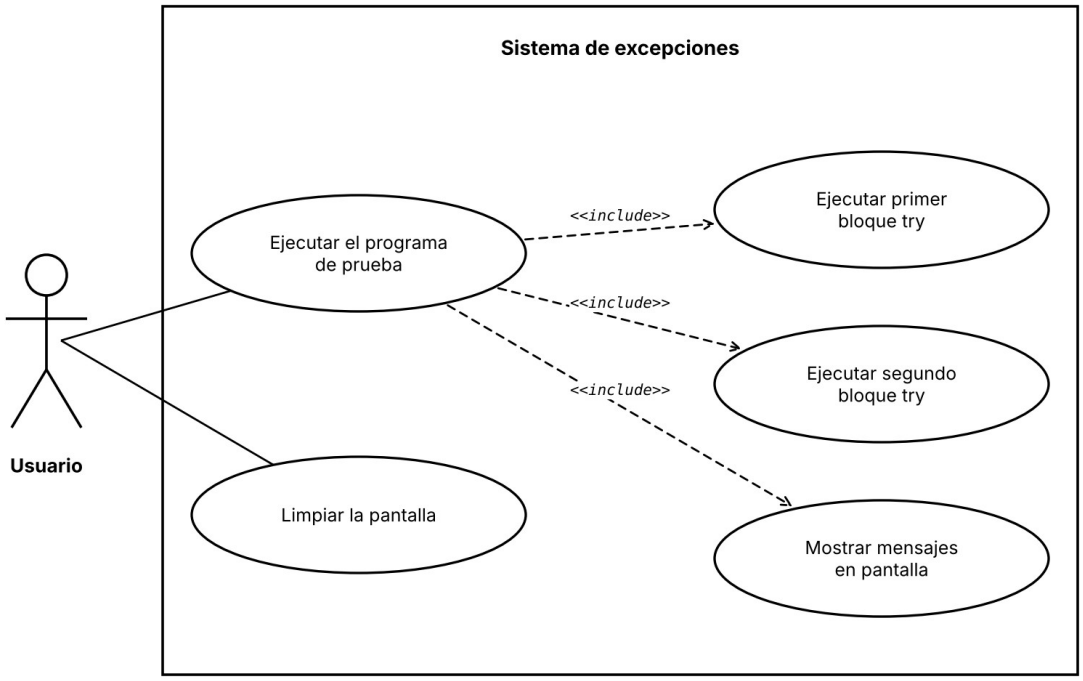
a) Resultado de la ejecución (botón Ejecutar)

Los mensajes mostrados corresponden al orden de ejecución de los bloques try, catch y finally.

Diagrama de Clase:



Casos de uso:



Caso de uso	Descripción
Ejecutar el programa de prueba	Al pulsar el botón Ejecutar el sistema corre los dos bloques try y muestra los mensajes en el cuadro de texto.
Ejecutar primer bloque try	Intenta una división por cero; la excepción aritmética es capturada y luego se ejecuta el finally.
Ejecutar segundo bloque try	Intenta usar un objeto nulo; la excepción general es capturada y luego se ejecuta el finally.
Mostrar mensajes en pantalla	Muestra en orden los mensajes generados durante la ejecución.
Limpiar la pantalla	Borra el contenido del cuadro de texto.

URL en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2/blob/c7409a279f6888247422a7b1c9c4a7cb5bdb44a3/principal_excepciones.py

Ejercicio 6.5 – Vendedor (pág. 406)

Enunciado: Se requiere implementar una clase Vendedor con los atributos nombre, apellidos y edad, un constructor, un método para imprimir sus atributos y un método para verificar la edad. Para desempeñar sus labores el vendedor debe ser mayor de 18 años; si no se cumple se lanza una excepción con el mensaje "El vendedor debe ser mayor de 18 años". Además, la edad debe estar entre 0 y 120; si no, se lanza una excepción con el mensaje "La edad no puede ser negativa ni mayor a 120". Los datos del vendedor se ingresan por la interfaz gráfica. (En Python se usa ValueError, equivalente a la IllegalArgumentException de Java.)

Archivo 1: vendedor.py

```
class Vendedor:
    def __init__(self, nombre, apellidos):
        self.nombre = nombre
        self.apellidos = apellidos
        self.edad = 0

    def imprimir(self):
        # devuelve el texto para mostrarlo en la ventana
        return (f"Nombre del vendedor = {self.nombre}\n"
                f"Apellidos del vendedor = {self.apellidos}\n"
                f"Edad del vendedor = {self.edad}")

    def verificar_edad(self, edad):
        # la edad tiene que estar entre 0 y 120
        if edad < 0 or edad > 120:
            raise ValueError("La edad no puede ser negativa ni mayor a 120.")
        # y el vendedor tiene que ser mayor de edad
        if edad < 18:
            raise ValueError("El vendedor debe ser mayor de 18 años.")
        self.edad = edad
```

Archivo 2: ventana_vendedor.py

```
import tkinter as tk
from tkinter import messagebox
from vendedor import Vendedor


class VentanaVendedor(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Vendedor")
        self.resizable(False, False)

        tk.Label(self, text="Nombre:", anchor="w").place(x=20, y=20, width=80, height=23)
        self.campo_nombre = tk.Entry(self)
        self.campo_nombre.place(x=105, y=20, width=190, height=23)

        tk.Label(self, text="Apellidos:", anchor="w").place(x=20, y=50, width=80, height=23)
```

```
        self.campo_apellidos = tk.Entry(self)
        self.campo_apellidos.place(x=105, y=50, width=190, height=23)

        tk.Label(self, text="Edad:", anchor="w").place(x=20, y=80, width=80, height=23)
        self.campo_edad = tk.Entry(self)
        self.campo_edad.place(x=105, y=80, width=190, height=23)

        self.boton_crear = tk.Button(self, text="Crear vendedor", command=self.crear_vendedor)
        self.boton_crear.place(x=105, y=115, width=120, height=25)

        # etiqueta donde se muestran los datos del vendedor
        self.resultado = tk.Label(self, text="", anchor="nw", justify="left")
        self.resultado.place(x=20, y=160, width=290, height=70)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 330) // 2
        y = (self.winfo_screenheight() - 250) // 2
        self.geometry(f"330x250+{x}+{y}")

    # evento del boton Crear vendedor
    def crear_vendedor(self):
        nombre = self.campo_nombre.get().strip()
        apellidos = self.campo_apellidos.get().strip()

        if nombre == "" or apellidos == "":
            messagebox.showwarning("Alerta", "Debe ingresar el nombre y los apellidos.")
            return

        try:
            edad = int(self.campo_edad.get())
        except ValueError:
            messagebox.showerror("Error", "La edad debe ser un número entero.")
            return

        vendedor = Vendedor(nombre, apellidos)
        try:
            vendedor.verificar_edad(edad)
        except ValueError as e:
            # si la edad no sirve se muestra la excepcion
            self.resultado.config(text="")
            messagebox.showerror("Excepción", str(e))
            return

        self.resultado.config(text=vendedor.imprimir())
```

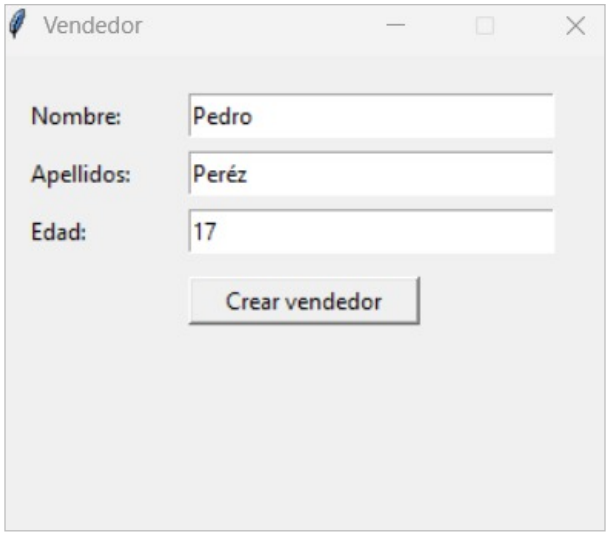
Archivo 3: principal_vendedor.py

```
from ventana_vendedor import VentanaVendedor
```

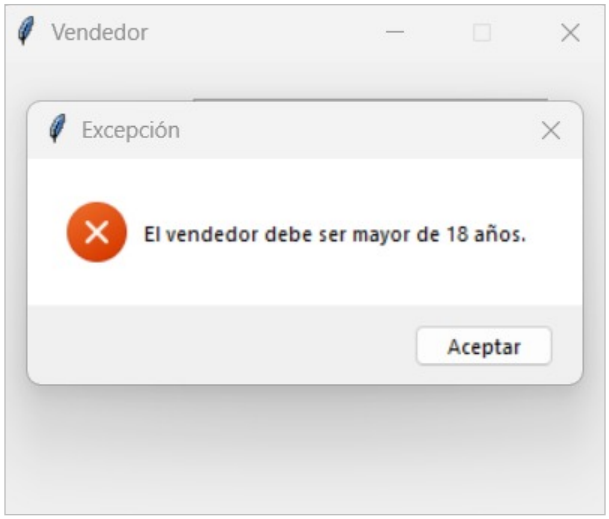
```
class Principal:
    def main():
        ventana = VentanaVendedor()
        ventana.mainloop()


if __name__ == "__main__":
    Principal.main()
```

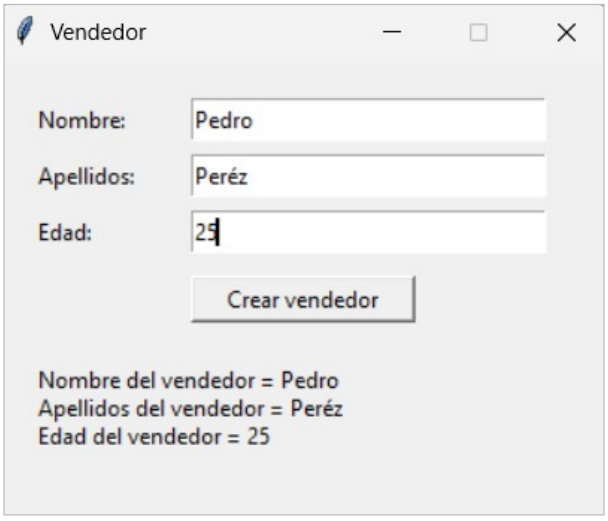
Interfaz de usuario:



a) Datos ingresados: edad 17



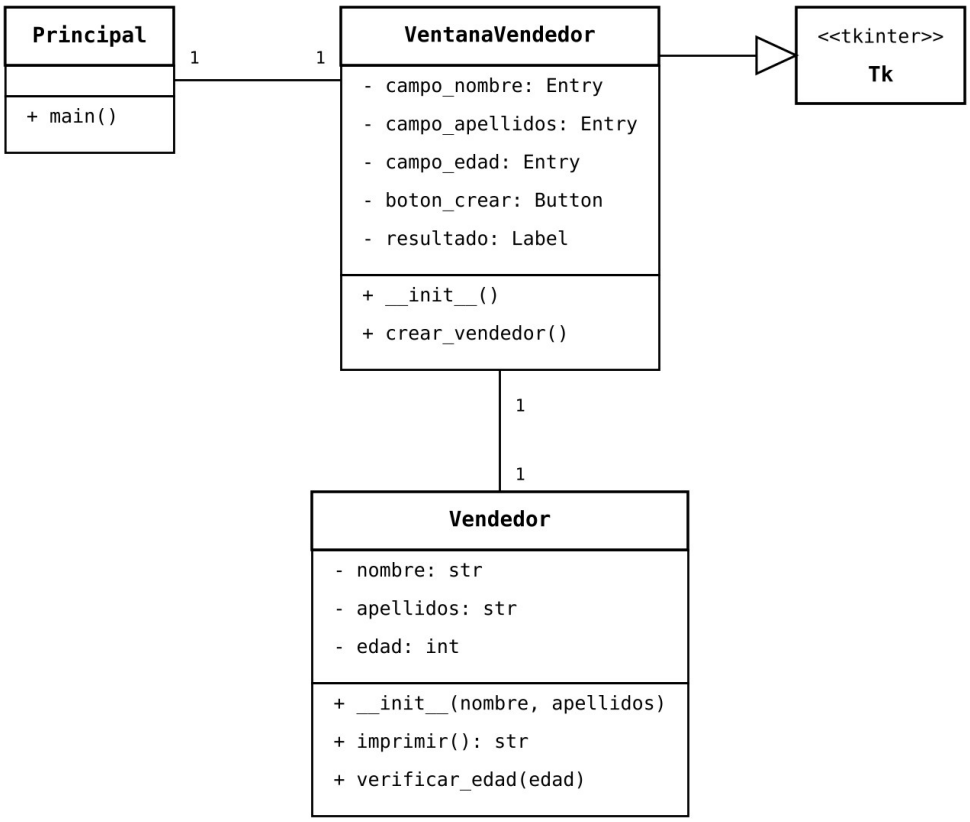
a) Excepción: edad menor de 18 años



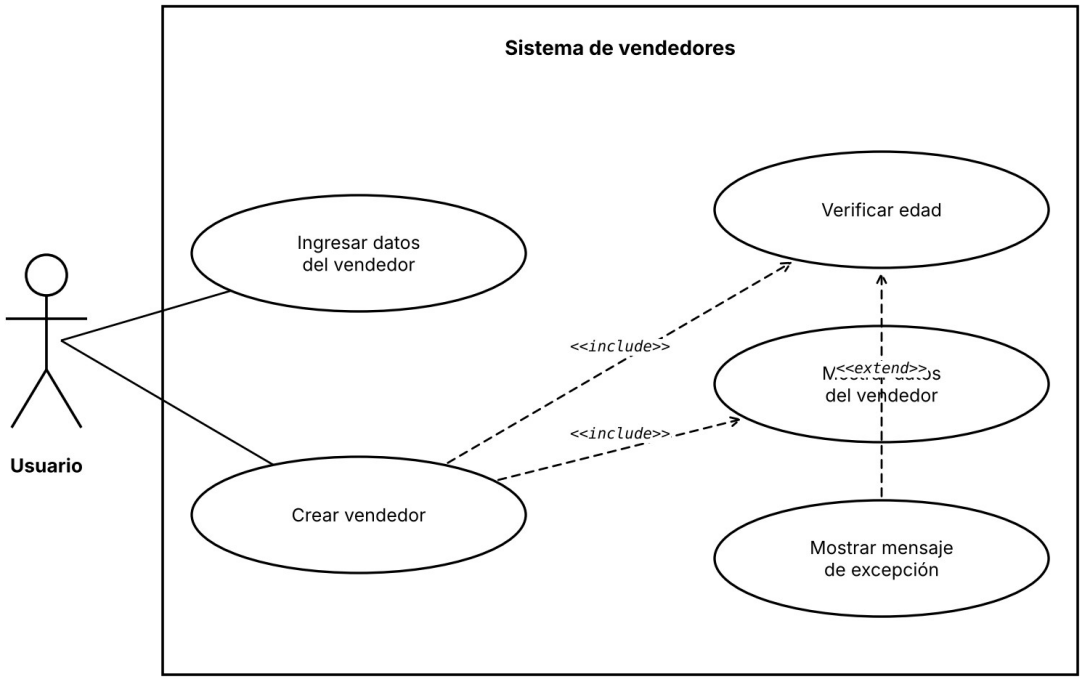
b) Vendedor creado correctamente (edad 25)

Datos de ejemplo: nombre Pedro, apellidos Pérez, con edad 17 (excepción) y con edad 25 (vendedor válido).

Diagrama de Clase:



Casos de uso:



Caso de uso	Descripción
Ingresar datos del vendedor	El usuario escribe el nombre, los apellidos y la edad del vendedor.
Crear vendedor	Al pulsar el botón Crear vendedor se crea el objeto y se verifica su edad.
Verificar edad	La edad debe estar entre 0 y 120 y el vendedor debe ser mayor de 18 años; si no, se lanza una excepción.
Mostrar datos del vendedor	Si la edad es válida se muestran el nombre, los apellidos y la edad.
Mostrar mensaje de excepción	Muestra el mensaje de la excepción cuando la edad no es válida o no es un número entero.

URL en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2/blob/c7409a279f6888247422a7b1c9c4a7cb5bdb44a3/principal_vendedor.py

Ejercicio 6.6 – Cálculos numéricos (pág. 412)

Enunciado: Se requiere definir una clase CálculosNuméricos que calcule el logaritmo neperiano y la raíz cuadrada de un valor recibido como parámetro (métodos estáticos). Si el valor no es positivo se genera una excepción aritmética. El valor se ingresa por la interfaz gráfica y se manejan las excepciones correspondientes (valor no numérico y valor no positivo).

Archivo 1: calculos_numericos.py

```
import math


class CalculosNumericos:
    @staticmethod
    def calcular_logaritmo_neperiano(valor):
        # si el valor no es positivo se lanza una excepcion aritmetica
        if valor <= 0:
            raise ArithmeticError("El valor debe ser un número positivo para calcular el logaritmo")
        return math.log(valor)

    @staticmethod
    def calcular_raiz_cuadrada(valor):
        if valor <= 0:
            raise ArithmeticError("El valor debe ser un número positivo para calcular la raíz
cuadrada")
        return math.sqrt(valor)
```

Archivo 2: ventana_calculos.py

```
import tkinter as tk
from tkinter import messagebox
from calculos_numericos import CalculosNumericos


class VentanaCalculos(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Cálculos numéricos")
        self.resizable(False, False)

        tk.Label(self, text="Valor numérico:", anchor="w").place(x=20, y=20, width=100, height=23)
        self.campo_valor = tk.Entry(self)
        self.campo_valor.place(x=125, y=20, width=150, height=23)

        self.boton_logaritmo = tk.Button(self, text="Logaritmo neperiano", command=self.logaritmo)
        self.boton_logaritmo.place(x=20, y=60, width=140, height=25)
        self.boton_raiz = tk.Button(self, text="Raíz cuadrada", command=self.raiz)
        self.boton_raiz.place(x=170, y=60, width=105, height=25)

        self.resultado = tk.Label(self, text="Resultado = ", anchor="w")
        self.resultado.place(x=20, y=105, width=260, height=23)
```

```
        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 300) // 2
        y = (self.winfo_screenheight() - 150) // 2
        self.geometry(f"300x150+{x}+{y}")

    # eventos de los botones
    def logaritmo(self):
        self.calcular(CalculosNumericos.calcular_logaritmo_neperiano)

    def raiz(self):
        self.calcular(CalculosNumericos.calcular_raiz_cuadrada)

    def calcular(self, operacion):
        self.resultado.config(text="Resultado = ")

        try:
            valor = float(self.campo_valor.get().replace(",", "."))
        except ValueError:
            messagebox.showerror("Error", "El valor debe ser numérico para realizar el cálculo")
            return

        try:
            resultado = operacion(valor)
            self.resultado.config(text=f"Resultado = {resultado:.4f}")
        except ArithmeticError as e:
            messagebox.showerror("Excepción aritmética", str(e))
```

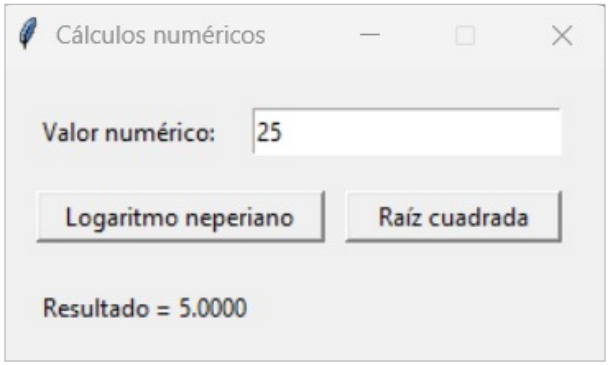
Archivo 3: principal_calculos.py

```
from ventana_calculos import VentanaCalculos

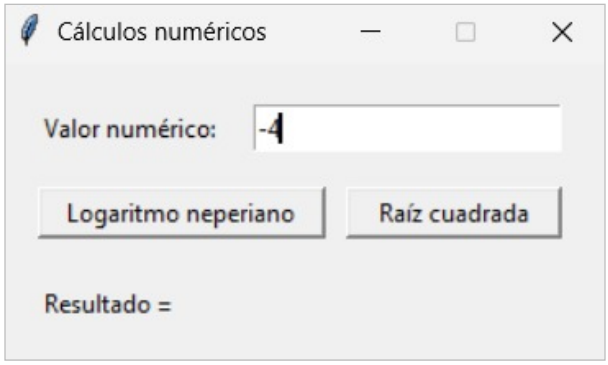

class Principal:
    def main():
        ventana = VentanaCalculos()
        ventana.mainloop()


if __name__ == "__main__":
    Principal.main()
```

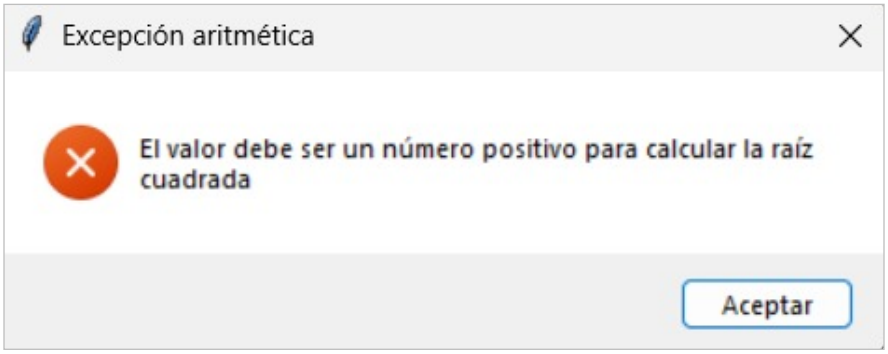
Interfaz de usuario:



a) Cálculo correcto (raíz cuadrada de 25)



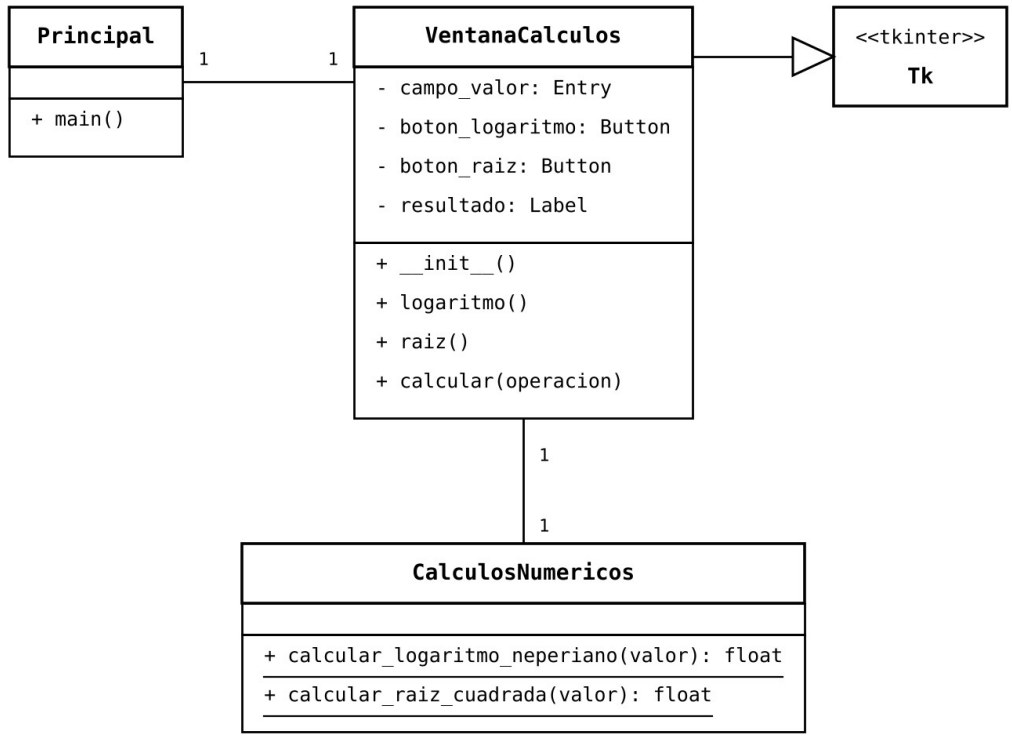
b) Valor -4 ingresado



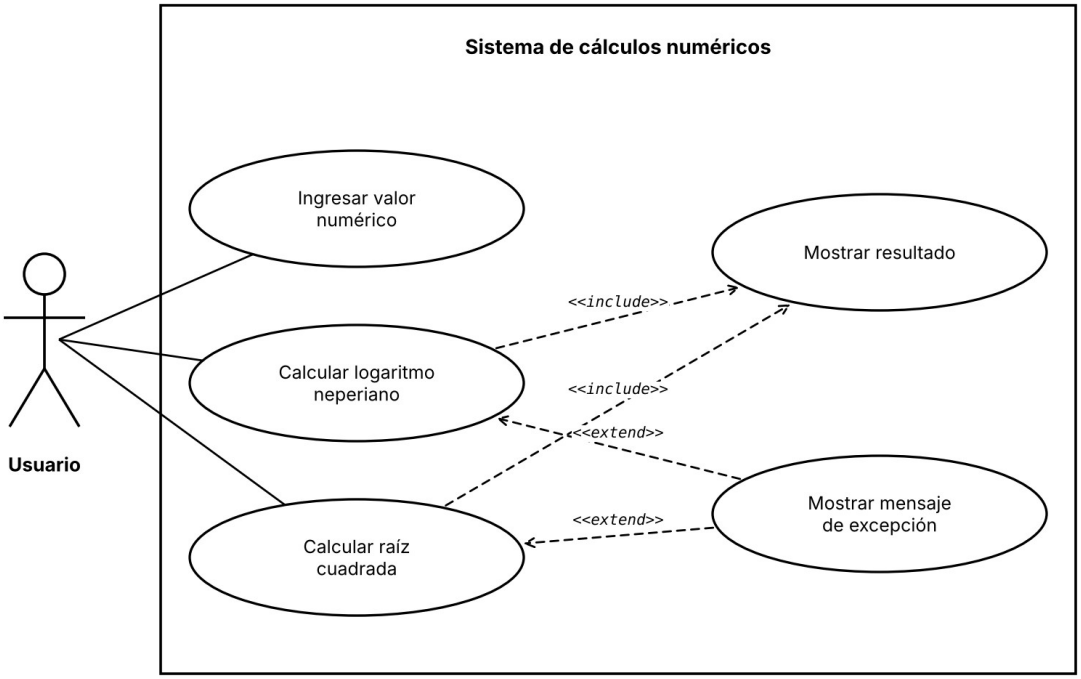
b) Excepción aritmética (valor -4)

Datos de ejemplo: valor 25 → raíz cuadrada 5.0000 y logaritmo neperiano 3.2189; el valor -4 genera la excepción.

Diagrama de Clase:



Casos de uso:



Caso de uso	Descripción
Ingresar valor numérico	El usuario escribe el valor en el campo de texto.
Calcular logaritmo neperiano	Calcula el logaritmo neperiano del valor; si no es positivo se lanza una excepción aritmética.
Calcular raíz cuadrada	Calcula la raíz cuadrada del valor; si no es positivo se lanza una excepción aritmética.
Mostrar resultado	Muestra el resultado del cálculo en la ventana.
Mostrar mensaje de excepción	Se muestra cuando el valor no es numérico o no es positivo.

URL en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2/blob/c7409a279f6888247422a7b1c9c4a7cb5bdb44a3/principal_calculos.py

Ejercicio 6.7 – Equipo de maratón de programación (pág. 418)

Enunciado: Un equipo de programadores desea participar en una maratón de programación. El equipo tiene nombre, universidad que representa, lenguaje de programación y tamaño, y está conformado por mínimo dos y máximo tres programadores (cada uno con nombre y apellidos). Se requiere un constructor, un método para determinar si el equipo está completo, un método para añadir programadores (si el equipo está lleno se genera la excepción correspondiente) y un método para validar el nombre y los apellidos de un programador: solo texto (si hay números se genera una excepción) y con longitud menor a 20 caracteres. Los datos se ingresan por la interfaz gráfica.

Archivo 1: programador.py

```
class Programador:
    def __init__(self, nombre, apellidos):
        self.nombre = nombre
        self.apellidos = apellidos
```

Archivo 2: equipo_maraton.py

```
from programador import Programador


class EquipoMaraton:
    def __init__(self, nombre_equipo, universidad, lenguaje):
        self.nombre_equipo = nombre_equipo
        self.universidad = universidad
        self.lenguaje = lenguaje
        self.programadores = []
        # el tamaño del equipo empieza en cero
        self.tamano_equipo = 0

    def esta_lleno(self):
        # maximo 3 programadores
        return self.tamano_equipo == 3

    def esta_completo(self):
        # minimo 2 programadores para poder competir
        return self.tamano_equipo >= 2

    def anadir(self, programador):
        if self.esta_lleno():
            raise Exception("El equipo está completo. No se pudo agregar programador.")
        self.programadores.append(programador)
        self.tamano_equipo += 1

    @staticmethod
    def validar_campo(campo):
        if campo == "":
            raise ValueError("El campo no puede estar vacío.")
        for c in campo:
            if c.isdigit():
```

```
                raise ValueError("El nombre no puede tener dígitos.")
        if len(campo) >= 20:
            raise ValueError("La longitud debe ser inferior a 20 caracteres.")
```

Archivo 3: ventana_equipo.py

```
import tkinter as tk
from tkinter import messagebox
from programador import Programador
from equipo_maraton import EquipoMaraton


class VentanaEquipo(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Equipo maratón de programación")
        self.resizable(False, False)
        self.equipo = None

        # datos del equipo
        tk.Label(self, text="Datos del equipo", anchor="w", font=("Segoe UI", 9,
"bold")).place(x=20, y=10, width=200, height=23)
        tk.Label(self, text="Nombre del equipo:", anchor="w").place(x=20, y=40, width=130,
height=23)
        self.campo_equipo = tk.Entry(self)
        self.campo_equipo.place(x=155, y=40, width=190, height=23)
        tk.Label(self, text="Universidad:", anchor="w").place(x=20, y=70, width=130, height=23)
        self.campo_universidad = tk.Entry(self)
        self.campo_universidad.place(x=155, y=70, width=190, height=23)
        tk.Label(self, text="Lenguaje:", anchor="w").place(x=20, y=100, width=130, height=23)
        self.campo_lenguaje = tk.Entry(self)
        self.campo_lenguaje.place(x=155, y=100, width=190, height=23)
        self.boton_crear = tk.Button(self, text="Crear equipo", command=self.crear_equipo)
        self.boton_crear.place(x=155, y=135, width=120, height=25)

        # datos del programador
        tk.Label(self, text="Datos del programador", anchor="w", font=("Segoe UI", 9,
"bold")).place(x=20, y=180, width=200, height=23)
        tk.Label(self, text="Nombre:", anchor="w").place(x=20, y=210, width=130, height=23)
        self.campo_nombre = tk.Entry(self)
        self.campo_nombre.place(x=155, y=210, width=190, height=23)
        tk.Label(self, text="Apellidos:", anchor="w").place(x=20, y=240, width=130, height=23)
        self.campo_apellidos = tk.Entry(self)
        self.campo_apellidos.place(x=155, y=240, width=190, height=23)
        self.boton_anadir = tk.Button(self, text="Añadir programador",
command=self.anadir_programador)
        self.boton_anadir.place(x=155, y=275, width=140, height=25)

        # estado del equipo e integrantes
        self.estado = tk.Label(self, text="Integrantes: 0 de 3 (mínimo 2)", anchor="w")
```

```
        self.estado.place(x=20, y=320, width=340, height=23)
        self.texto = tk.Text(self, state="disabled")
        self.texto.place(x=20, y=350, width=340, height=90)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 380) // 2
        y = (self.winfo_screenheight() - 460) // 2
        self.geometry(f"380x460+{x}+{y}")

    # evento del boton Crear equipo
    def crear_equipo(self):
        nombre = self.campo_equipo.get().strip()
        universidad = self.campo_universidad.get().strip()
        lenguaje = self.campo_lenguaje.get().strip()

        if nombre == "" or universidad == "" or lenguaje == "":
            messagebox.showwarning("Alerta", "Debe ingresar el nombre, la universidad y el
lenguaje.")
            return

        self.equipo = EquipoMaraton(nombre, universidad, lenguaje)
        self.actualizar()

    # evento del boton Añadir programador
    def anadir_programador(self):
        if self.equipo is None:
            messagebox.showwarning("Alerta", "Primero debe crear el equipo.")
            return

        nombre = self.campo_nombre.get().strip()
        apellidos = self.campo_apellidos.get().strip()

        try:
            # se validan el nombre y los apellidos
            EquipoMaraton.validar_campo(nombre)
            EquipoMaraton.validar_campo(apellidos)
            programador = Programador(nombre, apellidos)
            self.equipo.anadir(programador)
        except ValueError as e:
            messagebox.showerror("Excepción", str(e))
            return
        except Exception as e:
            messagebox.showerror("Excepción", str(e))
            return

        self.campo_nombre.delete(0, tk.END)
        self.campo_apellidos.delete(0, tk.END)
        self.actualizar()

    def actualizar(self):
```

```
        # muestra el estado del equipo y la lista de integrantes
        texto_estado = f"Integrantes: {self.equipo.tamano_equipo} de 3 (mínimo 2)"
        if self.equipo.esta_lleno():
            texto_estado += " - equipo lleno"
        elif self.equipo.esta_completo():
            texto_estado += " - equipo completo"
        self.estado.config(text=texto_estado)

        lineas = [f"Equipo: {self.equipo.nombre_equipo} ({self.equipo.universidad}) -
{self.equipo.lenguaje}"]
        for i in range(self.equipo.tamano_equipo):
            p = self.equipo.programadores[i]
            lineas.append(f"{i + 1}. {p.nombre} {p.apellidos}")

        self.texto.config(state="normal")
        self.texto.delete("1.0", tk.END)
        self.texto.insert(tk.END, "\n".join(lineas))
        self.texto.config(state="disabled")
```

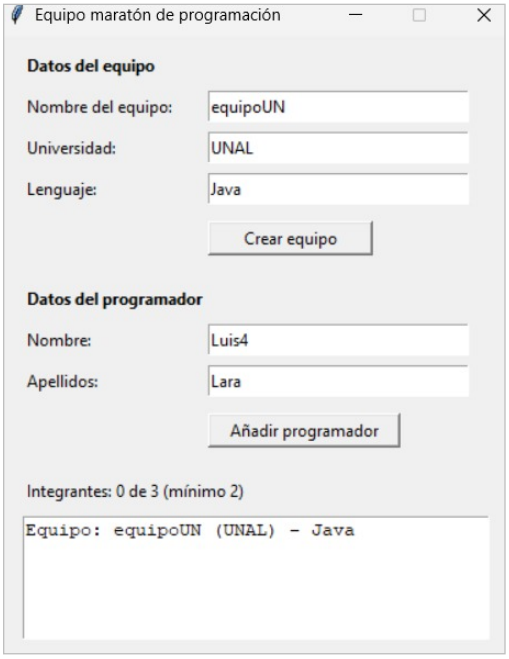
Archivo 4: principal_equipo.py

```
from ventana_equipo import VentanaEquipo

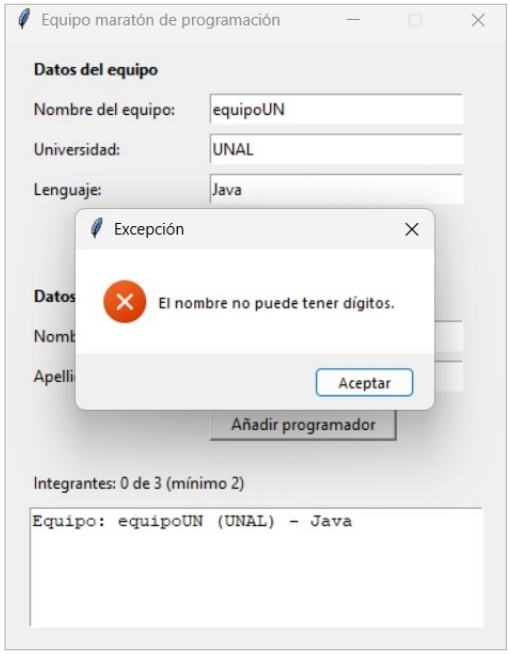

class Principal:
    def main():
        ventana = VentanaEquipo()
        ventana.mainloop()


if __name__ == "__main__":
    Principal.main()
```

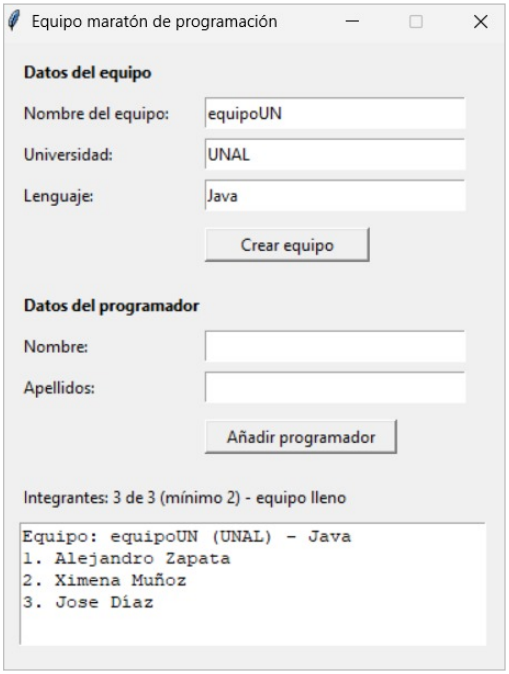
Interfaz de usuario:



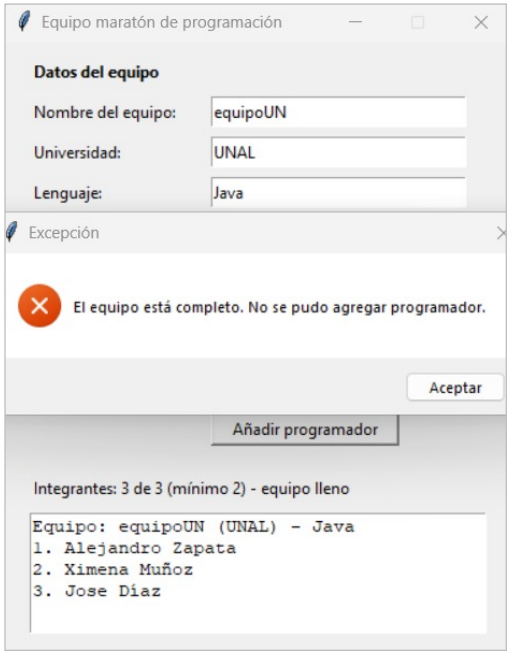
Equipo creado (equipoUN / UNAL / Java)



a) Excepción: nombre con dígitos



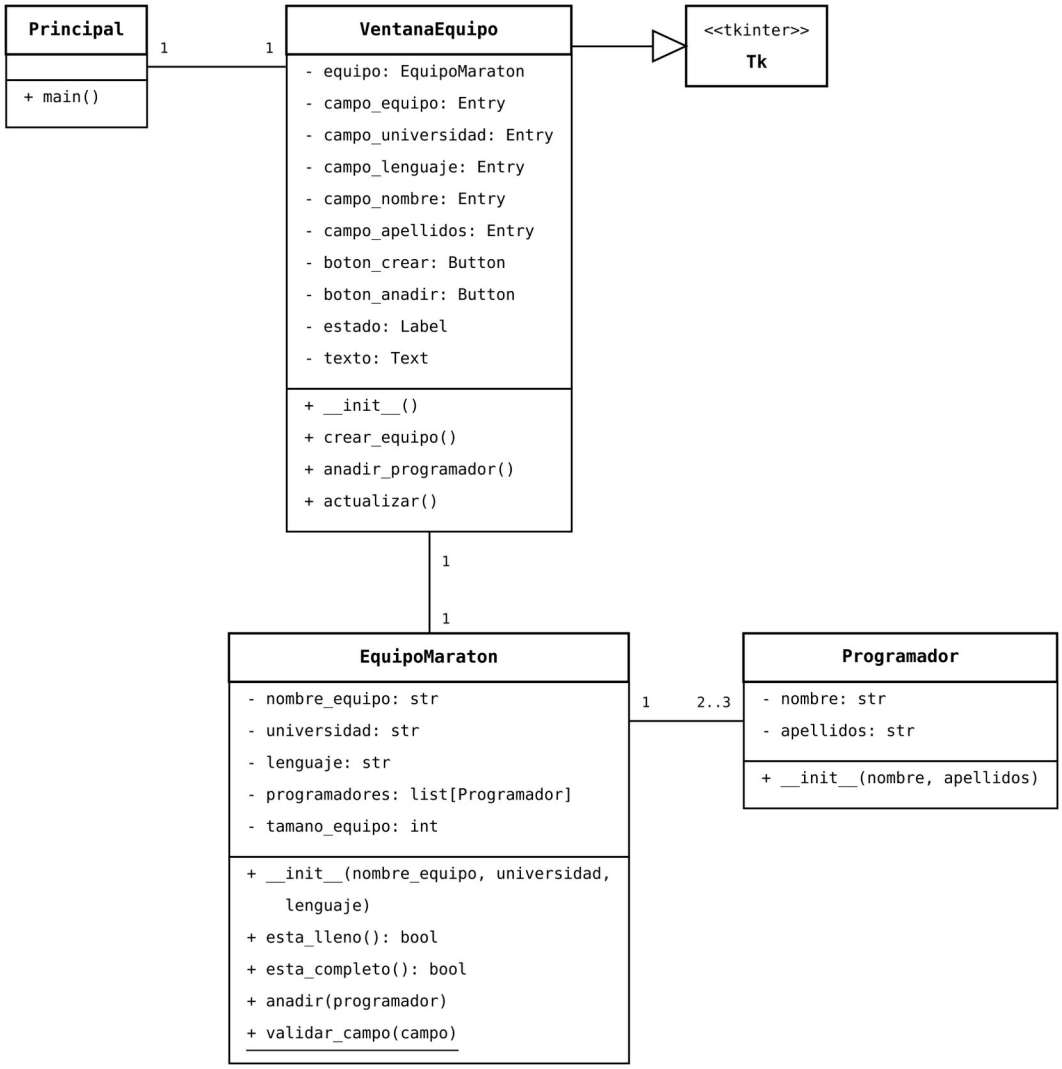
b) Equipo con tres integrantes



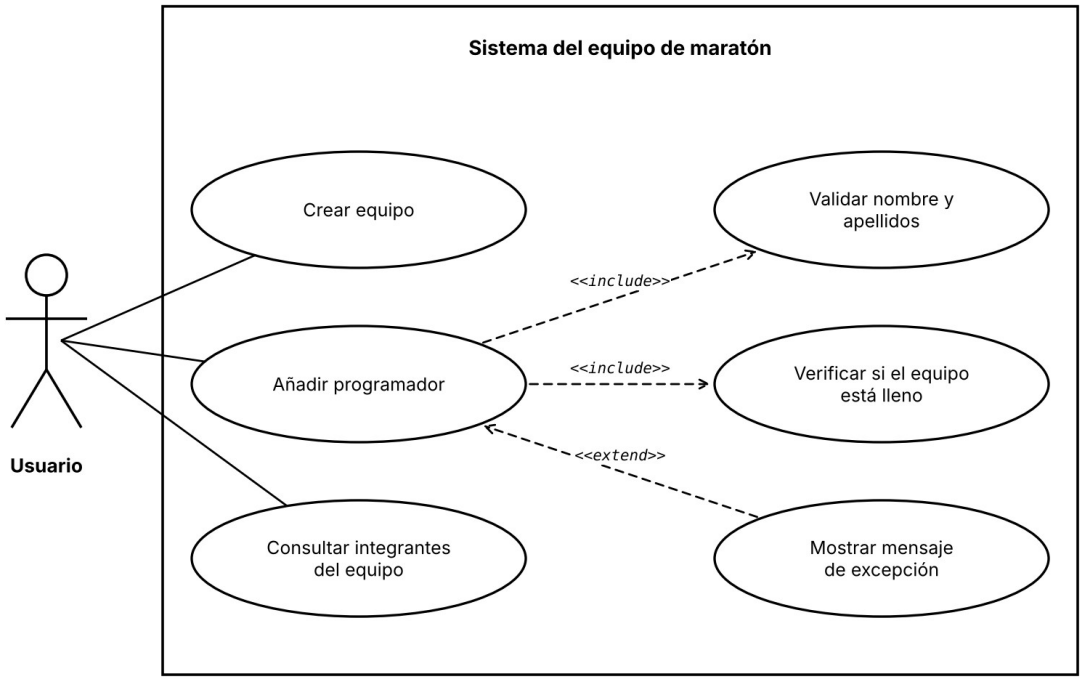
c) Excepción: equipo lleno

Datos de ejemplo: equipo equipoUN, universidad UNAL, lenguaje Java; programadores Alejandro Zapata, Ximena Muñoz y Jose Díaz; el nombre "Luis4" genera una excepción, y un cuarto programador también.

Diagrama de Clase:



Casos de uso:



Caso de uso	Descripción
Crear equipo	Crea el equipo con su nombre, universidad y lenguaje de programación.
Añadir programador	Valida el nombre y los apellidos y agrega el programador al equipo.
Validar nombre y apellidos	No deben estar vacíos, tener dígitos ni 20 o más caracteres; si no se cumple se lanza una excepción.
Verificar si el equipo está lleno	Un equipo admite máximo tres programadores; si está lleno se lanza una excepción.
Consultar integrantes del equipo	Muestra el estado del equipo y la lista de sus integrantes.
Mostrar mensaje de excepción	Muestra el mensaje de la excepción generada al añadir un programador.

URL en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2/blob/c7409a279f6888247422a7b1c9c4a7cb5bdb44a3/principal_equipo.py

Ejercicio 6.8 – Lectura de archivos (pág. 427)

Enunciado: Se tiene un archivo de texto denominado prueba.txt en una cierta localización del sistema de archivos. Se requiere desarrollar un programa que lea dicho archivo de texto utilizando un flujo de datos y muestre su contenido en pantalla (en esta versión, en la interfaz gráfica). Si no se puede leer el archivo se muestra el mensaje correspondiente.

Archivo 1: leer_archivo.py

```
class LeerArchivo:
    @staticmethod
    def leer(nombre_archivo):
        # se lee el archivo linea por linea
        lineas = []
        with open(nombre_archivo, encoding="utf-8") as archivo:
            linea = archivo.readline()
            while linea != "":
                lineas.append(linea.rstrip("\n"))
                linea = archivo.readline()
        return lineas
```

Archivo 2: ventana_archivo.py

```
import os
import tkinter as tk
from tkinter import filedialog, messagebox
from leer_archivo import LeerArchivo


class VentanaArchivo(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Leer archivo")
        self.resizable(False, False)

        tk.Label(self, text="Archivo:", anchor="w").place(x=20, y=20, width=60, height=23)
        self.campo_ruta = tk.Entry(self)
        self.campo_ruta.place(x=85, y=20, width=290, height=23)
        # por defecto se lee el prueba.txt
        self.campo_ruta.insert(0, "prueba.txt")

        self.boton_examinar = tk.Button(self, text="Examinar...", command=self.examinar)
        self.boton_examinar.place(x=385, y=19, width=90, height=25)
        self.boton_leer = tk.Button(self, text="Leer archivo", command=self.leer)
        self.boton_leer.place(x=20, y=55, width=120, height=25)

        # cuadro de texto con barra para mostrar el contenido
        self.texto = tk.Text(self, state="disabled")
        self.texto.place(x=20, y=95, width=435, height=190)
        barra = tk.Scrollbar(self, command=self.texto.yview)
        barra.place(x=455, y=95, width=20, height=190)
```

```
        self.texto.config(yscrollcommand=barra.set)

        # ventana centrada en la pantalla
        x = (self.winfo_screenwidth() - 495) // 2
        y = (self.winfo_screenheight() - 305) // 2
        self.geometry(f"495x305+{x}+{y}")

    # evento del boton Examinar
    def examinar(self):
        ruta = filedialog.askopenfilename(title="Seleccione un archivo de texto",
                                          filetypes=[("Archivos de texto", "*.txt"), ("Todos los
archivos", "*.*")])
        if ruta != "":
            self.campo_ruta.delete(0, tk.END)
            self.campo_ruta.insert(0, ruta)

    # evento del boton Leer archivo
    def leer(self):
        ruta = self.campo_ruta.get().strip()
        # si no se da la ruta completa se busca en la carpeta del programa
        if not os.path.isabs(ruta):
            ruta = os.path.join(os.path.dirname(os.path.abspath(__file__)), ruta)

        try:
            lineas = LeerArchivo.leer(ruta)
        except (OSError, UnicodeDecodeError):
            messagebox.showerror("Error", "No se pudo leer el archivo.")
            return

        self.texto.config(state="normal")
        self.texto.delete("1.0", tk.END)
        self.texto.insert(tk.END, "\n".join(lineas))
        self.texto.config(state="disabled")
```

Archivo 3: principal_archivo.py

```
from ventana_archivo import VentanaArchivo

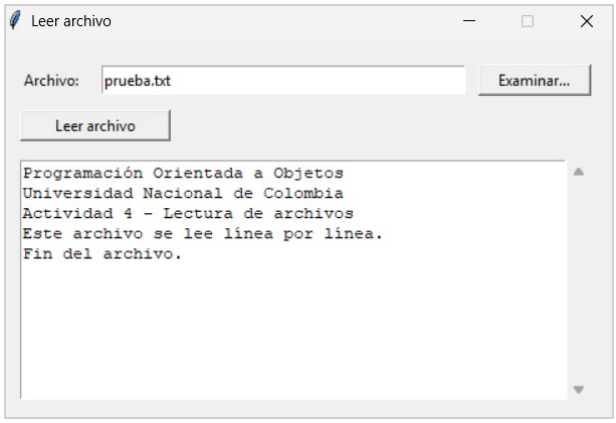

class Principal:
    def main():
        ventana = VentanaArchivo()
        ventana.mainloop()


if __name__ == "__main__":
    Principal.main()
```

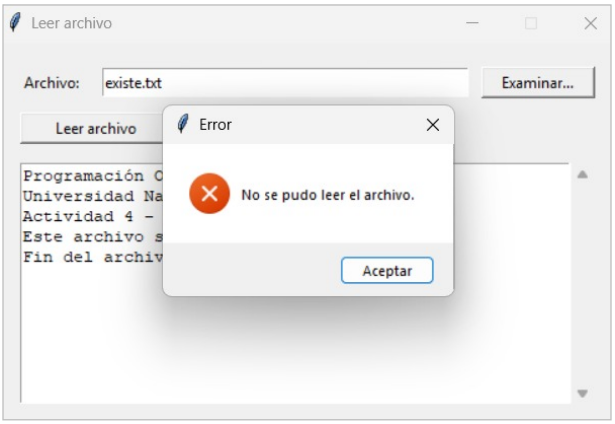
Archivo 4: prueba.txt

```
Programación Orientada a Objetos
Universidad Nacional de Colombia
Actividad 4 - Lectura de archivos
Este archivo se lee línea por línea.
Fin del archivo.
```

Interfaz de usuario:



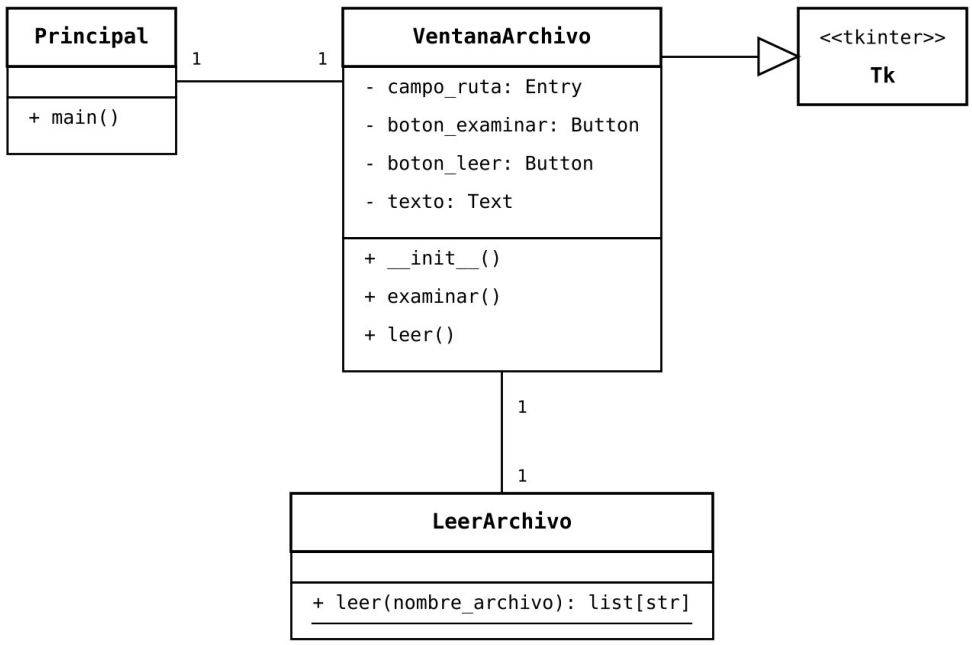
a) Contenido de prueba.txt



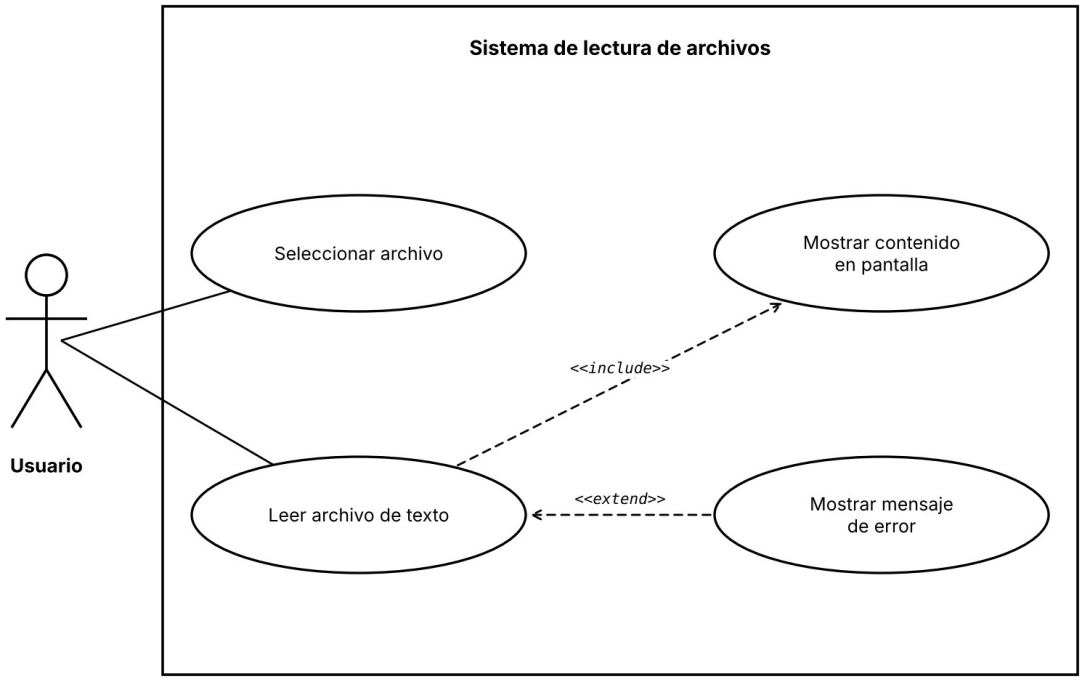
b) Archivo que no existe

Se lee el archivo prueba.txt incluido en el repositorio; un nombre de archivo inexistente muestra el mensaje de error.

Diagrama de Clase:



Casos de uso:



Caso de uso	Descripción
Seleccionar archivo	El usuario escribe la ruta del archivo o la busca con el botón Examinar.
Leer archivo de texto	Al pulsar el botón Leer archivo el sistema lee el archivo línea por línea.
Mostrar contenido en pantalla	Muestra en el cuadro de texto todas las líneas leídas.
Mostrar mensaje de error	Se muestra “No se pudo leer el archivo.” si el archivo no existe o no se puede leer.

URL en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2/blob/c7409a279f6888247422a7b1c9c4a7cb5bdb44a3/principal_archivo.py

URL del repositorio en GitHub:

https://github.com/xmunozl-collab/Actividad-4-POB-2026-2